Learning Minimal-Fuel Corrections Towards the Figure-8 Orbit

Ignacio Blancas Rodriguez [1] Ryan L Yang [1] Patrick Verma [1]

[1]Brown University

Problem statement

The N-body problem considers three planetary bodies and the evolution of their positions and velocities over time under their gravitational interactions. For $N \geq 3$ there is no closed-form solution and the solutions may become chaotic. We consider a 3-agent RL setup, where each agent is able to apply an a_i acceleration to the i-th body every ΔT time-step. The goal is for the agents to learn minimal a_i pushes to get the three bodies in some target stable orbit, for example, the Figure-8 orbit.

Architecture Description

In order for the independent actors to learn coordinated actions we train the independent agents via a centralized critic and reward function. We break our system down into the following layers,

- **Physics environment:** a numerical 3-body simulator updates positions and velocities using the agents' thrust actions and gravitational interactions.
- **Graph encoder:** A GNN, where each body corresponds to a node and edges encode their gravitational interactions, produces three embedding vectors h_i one for each body.
- **Decentralized actors:** Each actor is given an embedding h_i and produces an acceleration vector a_i that will be applied to the i-th body for ΔT time.
- **Centralized critic and reward:** During training, a global critic observes the full system state and joint action. The reward combines orbit-position error, velocity error, fuel penalty, survival rewards, phase synchronization, and stable permutation of the bodes.

The reward function

Our simulation implements the following reward function,

$$R = -\left(\omega_o R_o + \omega_f R_f + \omega_s R_s + \omega_p R_p + \omega_{\text{perm}} R_{\text{perm}}\right)$$

where each ω represents the weight of each sub-reward, and,

- $R_o = \frac{1}{3}\sum_{i=1}^{3}\left\|x_i - x_{\sigma_t(i)}^{\text{target}}\right\|^2 + \frac{1}{3}\sum_{i=1}^{3}\left\|1 - \frac{v_i \cdot v_{\sigma_t(i)}^{\text{target}}}{\|v_i\|\left\|v_{\sigma_t(i)}^{\text{target}}\right\| + 10^{-12}}\right\|^2$
- $R_f = \frac{1}{3}\sum_{i=1}^{3}\frac{\|a_i\|}{\text{max\_action\_norm}^2 + 10^{-12}}$
- $R_s = 1$ if there is a collision and 0 otherwise
- $R_p = \frac{\min(|t-\text{expected}|, N - |t-\text{expected}|)}{N}$
- $R_{\text{perm}} = \frac{\#\{i \in \{1,2,3\} \mid \sigma_{t-1}(i) \neq \sigma_t(i)\}}{3}$

Matching the state to the target orbit

We store the Figure-8 as reference snapshots: where each body should be and which direction it should move at each point in the choreography. Because the bodies have equal mass, their labels are interchangeable. At each step, the environment tries every assignment between simulated bodies and reference bodies, then chooses the best match.

The match uses four criteria:

- **Position:** bodies should be close to the reference locations.
- **Velocity direction:** bodies should move in the same direction as the reference choreography.
- **Phase continuity:** the matched point on the Figure-8 should move smoothly forward over time.
- **Assignment continuity:** the same simulated body should keep matching the same reference track unless switching gives a much better fit.

The assignment-continuity penalty prevents the matcher from rapidly swapping body identities between frames. This makes the reward reflect a cleaner physical choreography, where each body settles into a consistent role in the Figure-8 motion.

Initial Positions

Fully random starts made the task too difficult: most episodes quickly produced collisions, escapes, or states far from the Figure-8. We therefore use **directed initialization**: start from one fixed configuration near the target choreography, then add small random perturbations to positions and velocities.

This keeps training near the orbit we care about while still preventing the policy from memorizing one exact trajectory. The agents learn local corrective pushes that stabilize the Figure-8 while keeping fuel usage small.

Training and Challenges

One of the major struggles we found during training was the **signal strength** of our rewards function. With such a large action space as ours, the space of possible winning solutions is too large for the model to learn. We found that requiring stricter conditions to match the target orbit reduces the solution space and allows us to attain stronger training convergence.

Moreover, matching the current state of our system to a state in the target orbit proved complicated, as the naive approach of sampling points in the orbit and finding the one that is location-wise closest does not prevent weird artifacts where the bodies start to jitter and switch places.

Through experiments on a fixed position setup, we found that a training cycle of around ~ 2.5 hours achieves a total best reward of around ~ -104.

References

[1] V. S. Ulibarrena and S. P. Zwart. Reinforcement learning for adaptive time-stepping in the chaotic gravitational three-body problem. *Communications in Nonlinear Science and Numerical Simulation*, 145:108723, 2025.

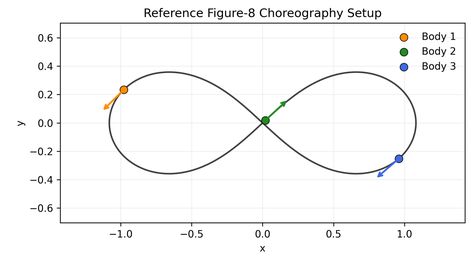

Figure 1. Reference Figure-8 choreography. The stored target orbit provides position and velocity targets for matching the simulated state at each step.

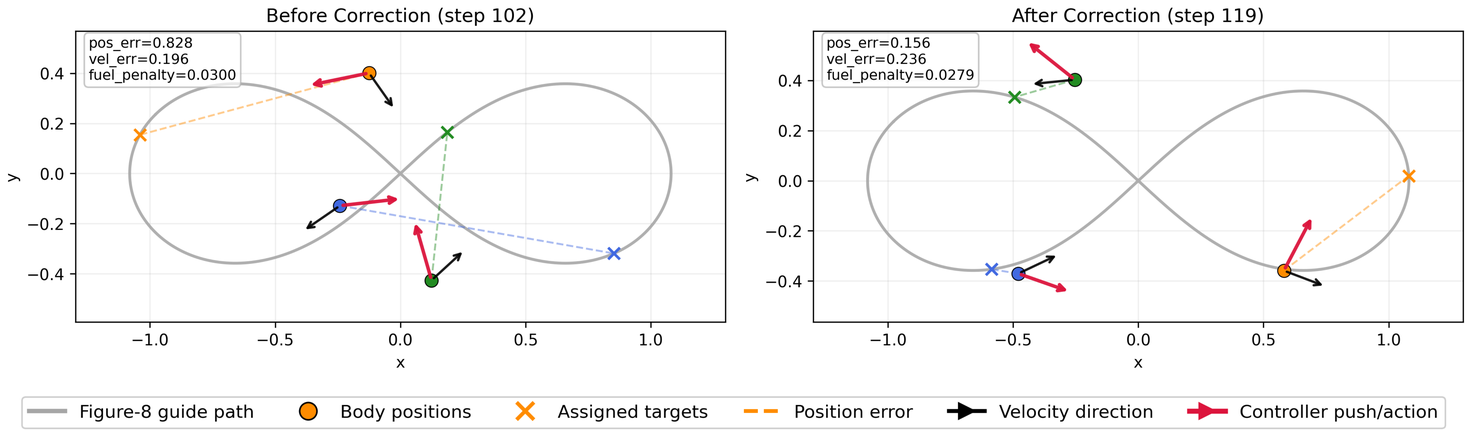

Figure 2. Directed initialization and policy correction. The policy learns small thrusts that move perturbed states toward the Figure-8 while penalizing larger fuel usage.